



DSA103 Advanced Chemical Data Science

Lecture 4: Python review part 2

Prof. Meredith C. Schuman

meredithchristine.schuman@uzh.ch

```
1 def search(self, state, g_score):
2     cube = RubiksCube(state=state)
3     if cube.solved():
4         return True
5     elif len(self.moves) >= self.threshold:
6         return False
7     min_val = float('inf')
8     best_action = None
9     for a in [(r, n, d) for r in ['h', 'v', 's'] for d in [0, 1] for n in range(cube.n)]:
10        cube = RubiksCube(state=state)
11        if a[0] == 'h':
12            cube.horizontal_twist(a[1], a[2])
13        elif a[0] == 'v':
14            cube.vertical_twist(a[1], a[2])
15        elif a[0] == 's':
16            cube.side_twist(a[1], a[2])
17        if cube.solved():
18            self.moves.append(a)
19            return True
20        cube_str = cube.stringify()
21        h_score = self.heuristic[cube_str] if cube_str in self.heuristic else self.max_depth
22        f_score = g_score + h_score
23        if f_score < min_val:
24            min_val = f_score
25            best_action = [(cube_str, a)]
26        elif f_score == min_val:
27            if best_action is None:
28                best_action = [(cube_str, a)]
29            else:
30                best_action.append((cube_str, a))
31    if best_action is not None:
32        if self.min_threshold is None or min_val < self.min_threshold:
33            self.min_threshold = min_val
34        next_action = choice(best_action)
35        self.moves.append(next_action[1])
36        status = self.search(next_action[0], g_score + min_val)
37        if status: return status
38    return False
```

Discussion of homework

Exercise 1: Molecular Formula Parser

Objective: Write a function `parse_molecular_formula(formula)` that counts atoms in molecular formulas. Start with the simple cases, but design your solution to handle increasingly complex inputs without major rewrites.

Think computationally: How can you break this problem into smaller, reusable pieces that won't need to be completely rewritten when the requirements change?

In small groups, discuss:

- Did you solve it?
- Did your solution work for all the examples or did it fail for some?
- What strategy did you use?

Then we'll recap together.

Molecular Formula Parser: A solution using RegEx and defaultdict

```
import re # This is needed in order to use RegEx (regular expressions)
from collections import defaultdict #defaultdict creates default entries and thus allows to add to existing entries

def convert_quantities(quantity : str):
    if len(quantity) == 0:
        return 1
    else:
        return int(quantity)

def parse_molecular_formula(formula: str) -> dict[str, int]:
    """
    Because there is "N", "I" and "Ni" in elements, to split this in a unique way we require the correct capitalization in the input.
    """
    atom_dict = defaultdict(int)
    pattern = r"([A-Z][a-z]?)([0-9]*)"

    components = re.findall(pattern, formula)
    for component in components:
        atom_dict[component[0]] += convert_quantities(component[1])

    return atom_dict
```

Molecular Formula Parser: A solution using a double while loop with two pointers

```
# Alternative Double while loop with two pointers (i, j):
# While 1: Loop over pairs of atom/element e.g. Cl2
# While 2: Loop over atom/element pair and extract element and number of atoms

from typing import Dict

def parse_molecular_formula(formula: str) -> Dict[str, int]:
    atom_dict = dict()

    i = 0
    while i < len(formula):
        extracted_element = ""
        extracted_n_atoms = ""

        j = 0
        while (j+i) < len(formula):
            # Break condition: Start of next element
            if extracted_element != "" and formula[i+j].lower() != formula[i+j]: # Alternative: extracted_element != "" and formula[i + j].isupper()
                break
            elif formula[i+j].isalpha():
                extracted_element += formula[i+j]
            elif formula[i+j].isnumeric():
                extracted_n_atoms += formula[i+j]
            j += 1

        if extracted_element in atom_dict:
            atom_dict[extracted_element] = atom_dict[extracted_element] + (int(extracted_n_atoms) if extracted_n_atoms != "" else 1)
        else:
            atom_dict[extracted_element] = int(extracted_n_atoms) if extracted_n_atoms != "" else 1

        i = i + j

    return atom_dict
```

Molecular Formula Parser

- Other solutions?
- What did you learn?
- Questions?



Chemical Equation Balance Checker

Exercise 2: Chemical Equation Balancer Checker

Objective: Write a program that checks if a chemical equation is balanced.

Problem: Create a function called `is_balanced()` that takes a chemical equation string and returns True if balanced, False if not.

Your Task:

- Split the equation at the arrow symbol "→" to get reactants and products
- For each side, parse the molecules and their coefficients
- Count total atoms of each element on both sides
- Compare the totals

Example Equation Format: "2H₂ + O₂ → 2H₂O"

In small groups, discuss:

- Did you solve it?
- Did your solution work for all the examples or did it fail for some?
- What strategy did you use?

Then we'll recap together.

Chemical Equation Balance Checker: A solution

Re-use functions from the Molecular Formula Parser

You need to deal with coefficients: add a function, e.g. `“apply_multiplier”`

Strategy (3-step example):

1. Split equations into reactants, products
2. Inventory atoms, apply coefficients
3. Check if the equation is balanced by checking amounts on each side

```
def apply_multiplier(atoms : dict, multiplier: int):  
    for atom, value in atoms.items():  
        atoms[atom] = value * multiplier  
    return atoms
```

✓ 0.0s

```
1 def split_equation(chem_eq : str, verbose=False):  
    left_side = chem_eq.split("→")[0]  
    reagents = left_side.split("+")  
    if verbose:  
        print("reagents: ")  
        print(reagents)  
  
    right_side = chem_eq.split("→")[1]  
    products = right_side.split("+")  
    if verbose:  
        print("products: ")  
        print(products)  
    return reagents, products  
  
2 def split_sides_into_amounts(chemicals: str):  
    chemical_dict = defaultdict(int)  
    pattern = r"([0-9]*)([A-Z][a-z])"  
  
    # Use convert_quantities and parse_molecular_formula functions defined in prior exercise above  
    for chemical in chemicals:  
        chemical = chemical.strip()  
        matches_r = re.findall(pattern, chemical)  
        multiplier_r = convert_quantities(matches_r[0][0])  
        chemical_atoms = parse_molecular_formula(chemical)  
        chemical_atoms = apply_multiplier(chemical_atoms, multiplier_r)  
        for atom_r, value_r in chemical_atoms.items():  
            chemical_dict[atom_r] += value_r  
  
    return chemical_dict  
  
3 def is_balanced(chem_eq : str, verbose=False):  
    reagents, products = split_equation(chem_eq, verbose)  
    return split_sides_into_amounts(reagents) == split_sides_into_amounts(products)  
  
chem_eq = "2H2 + O2 → CH4 → 2H2O + CH4"  
is_balanced(chem_eq)
```

✓ 0.0s

True

Reaction Pathway Analyzer

Exercise 3: Reaction Pathway Analyzer

Objective: Analyze multi-step reaction sequences to find synthesis pathways.

Problem: Write a program that determines what compounds can be synthesized from starting materials through a series of reactions, and finds a pathway to create a target compound.

Your Task: Create a function called `find_synthesis_path()` that takes:

- A list of starting compounds
- A list of reaction equations
- A target compound to synthesize

The function should return either:

- A list of reaction steps that lead to the target compound, OR
- A message saying the target cannot be synthesized

In small groups, discuss:

- Did you solve it?
- Did your solution work for all the examples or did it fail for some?
- What strategy did you use?

Then we'll recap together.

Reaction Pathway Analyzer: Two solutions

Insights:

- We can try all reactions from stock and add products to stock if we can do a reaction
- Once we have accessed a reaction, we don't need to access it again, but we may track the sequence in which it was accessed (path)
- Some reactions may be inaccessible or irrelevant to making the target

Two commonly used search strategies:

- Breadth first --> always return to stock and re-visit uncompleted reactions each round, then later re-construct the path that led to the target (if there is one)
- Depth first --> first follow the whole path and then backtrack (like wall-following)

Reaction Pathway Analyzer: Two solutions

```
# Both solutions require the following dataclass and helper functions (as well as import re)

from dataclasses import dataclass
# A dataclass is a nicely packaged object that keeps data together.

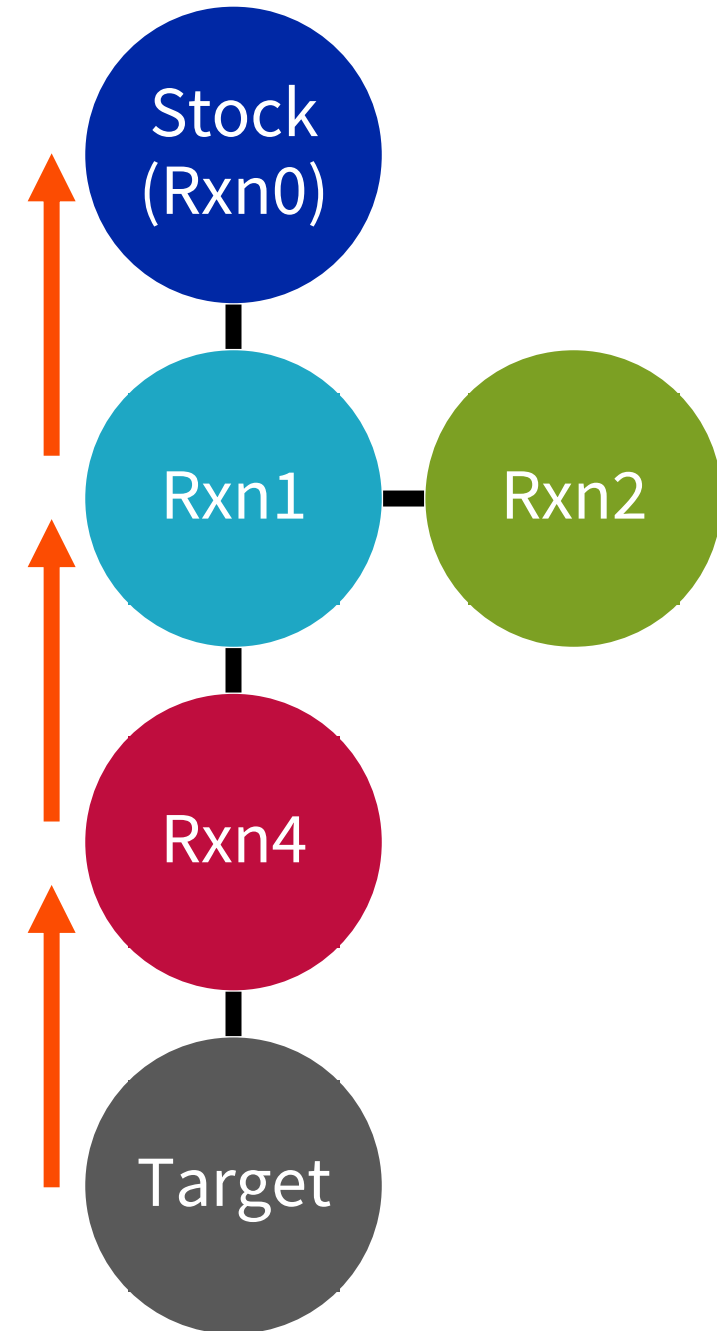
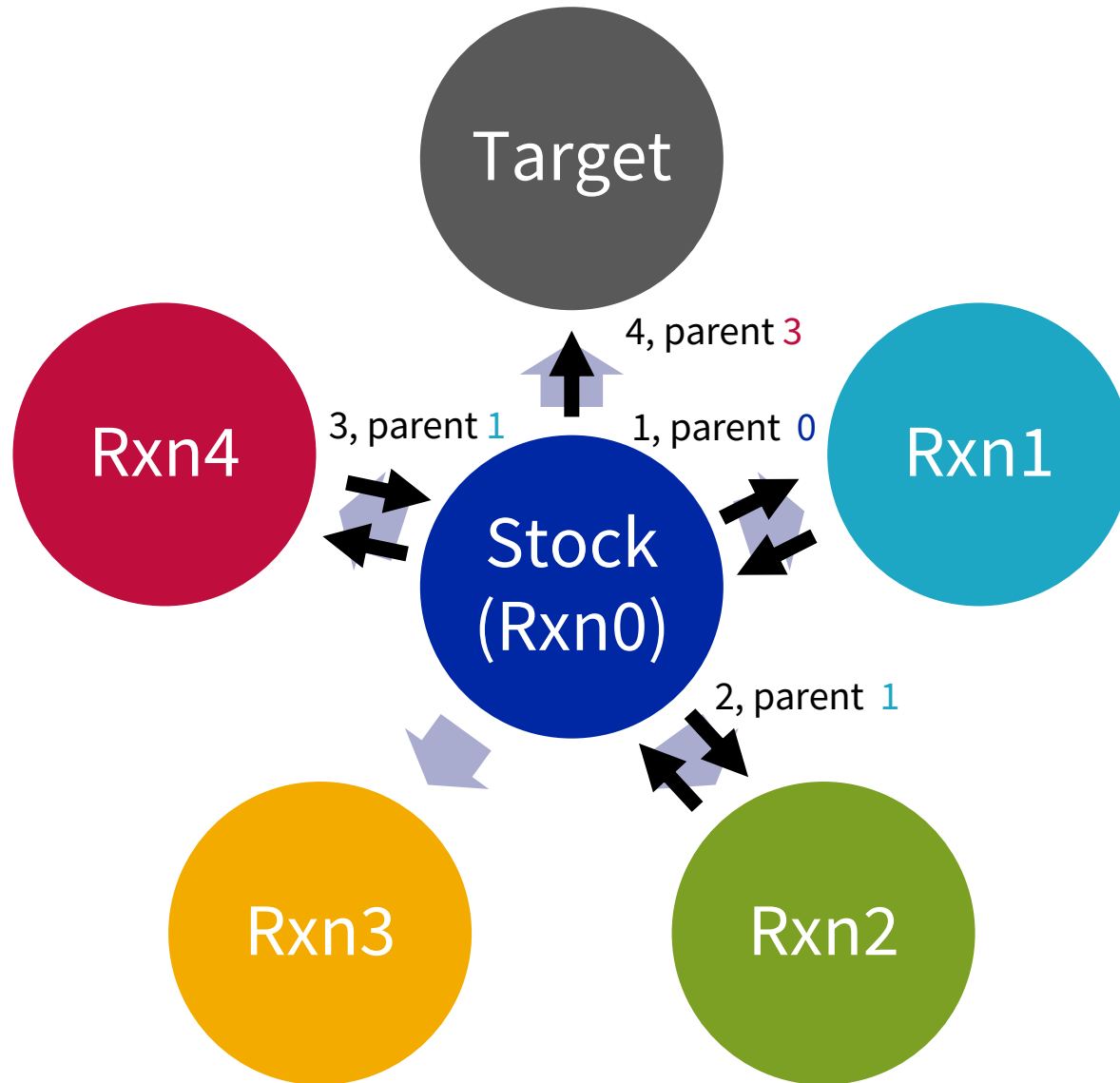
@dataclass
class Reaction:
    reagents : list[str]
    products : list[str]
    full_string : str
    order : int = 0
    parent = None
    used : bool = None

def split_molecules(molecules : str):
    pattern = r"[0-9]*([^\s+ ]+)" # an optional number, followed by the molecule which does not contain a space or `+` character
    return re.findall(pattern, molecules)

def split_reaction(reaction : str):
    reagents, products = reaction.split(">")
    reagent_molecules = split_molecules(reagents)
    product_molecules = split_molecules(products)

    return reagent_molecules, product_molecules
```

“Breadth first” solution



"breadth first" solution

```
def find_synthesis_path_broad(starting_materials: list[str], reactions: list[str], target: str) -> list[str] | str:
```

```
    parsed_reactions = []
```

```
    for reaction in reactions:
```

```
        reagent_molecules, product_molecules = split_reaction(reaction)
```

```
        parsed_reaction = Reaction(reagent_molecules, product_molecules, reaction, order=0)
```

```
        parsed_reactions.append(parsed_reaction)
```

```
    starting_materials = set(starting_materials)
```

```
    order = 1
```

```
    parent = "Stock"
```

```
    poof = True
```

```
    while poof:
```

```
        poof = False
```

```
        for parsed_reaction in parsed_reactions:
```

```
            if parsed_reaction.order != 0:
```

```
                # this has been used, do not use again
```

```
                continue
```

```
            if set(parsed_reaction.reagents).issubset(starting_materials):
```

```
                starting_materials = starting_materials.union(set(parsed_reaction.products))
```

```
                parsed_reaction.order = order
```

```
                parsed_reaction.parent = parent
```

```
                order += 1
```

```
                poof = True
```

```
                parent = parsed_reaction
```

```
                break
```

```
    if target in starting_materials:
```

```
        print("Target can be synthesized.")
```

```
        for parsed_reaction in parsed_reactions:
```

```
            if target in set(parsed_reaction.products):
```

```
                # print(parsed_reaction.full_string)
```

```
                # print(parsed_reaction.parent)
```

```
                parent_chain(parsed_reaction)
```

```
                break
```

```
    else:
```

```
        print("Target cannot be synthesized.")
```

```
def parent_chain(parsed_reaction : Reaction):
```

```
    if parsed_reaction != "Stock":
```

```
        print(parsed_reaction.full_string, end="")
```

```
        print(" <==> ", end="")
```

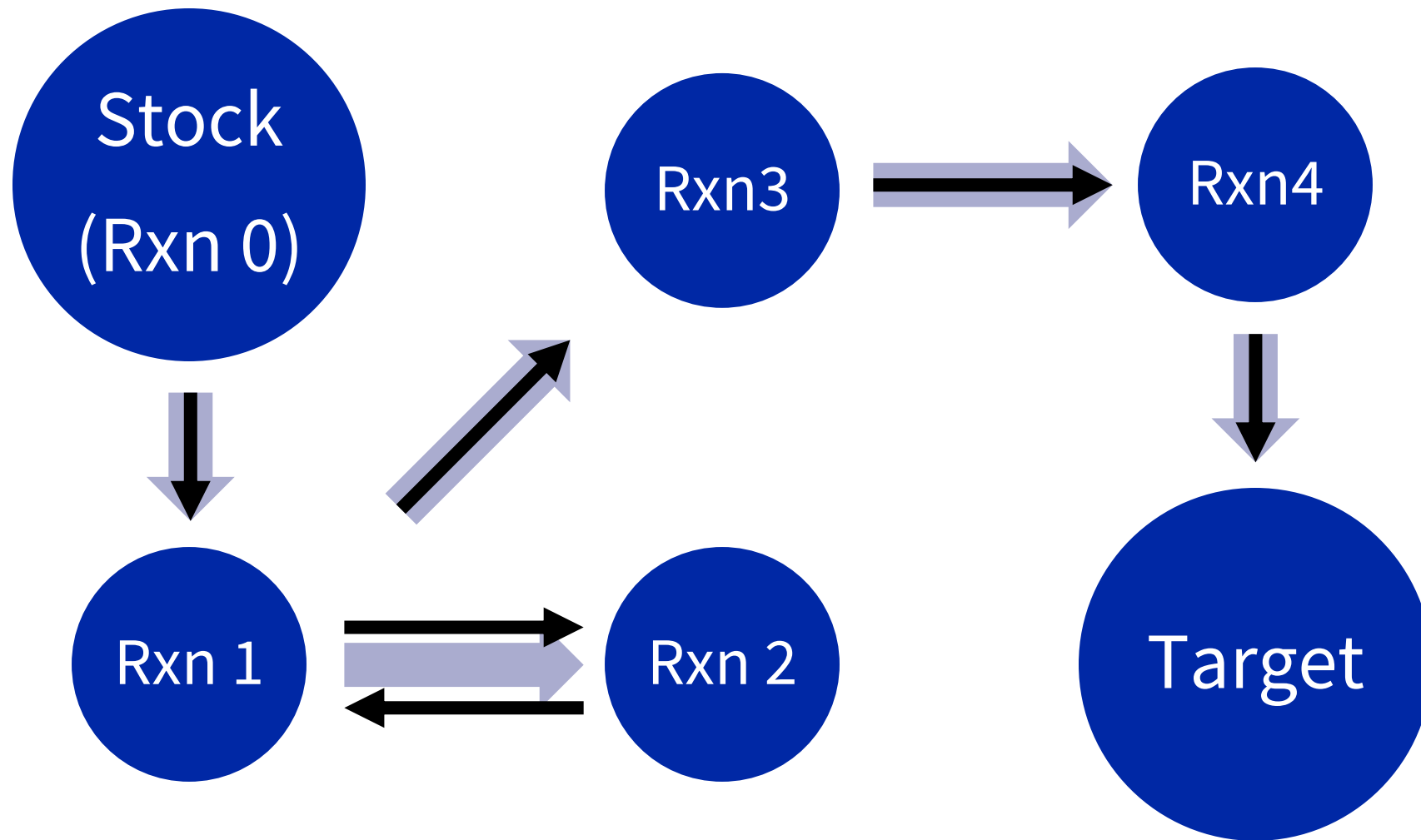
```
        if parsed_reaction.parent is not None:
```

```
            parent_chain(parsed_reaction.parent)
```

```
    elif parsed_reaction == "Stock":
```

```
        print("Stock")
```

“Depth first” solution



“Depth first” solution

```
# "depth first" solution

def find_synthesis_path_deep(starting_materials : list[str], reactions : list[str], target: str):

    reactions.insert(0, ">" + "+".join(starting_materials)) # Make stock look like reaction products for consistency

    parsed_reactions = []
    for reaction in reactions:
        reagent_molecules, product_molecules = split_reaction(reaction)
        parsed_reaction = Reaction(reagent_molecules, product_molecules, reaction,)
        parsed_reactions.append(parsed_reaction)

    if find_reaction(target, parsed_reactions): #This is recursive
        return True, [reaction.full_string for reaction in parsed_reactions if reaction.used]
    else:
        return False, []

def find_reaction(target: str, parsed_reactions: list[Reaction]):
    if (target == ""):
        return True

    for reaction in parsed_reactions:
        if target in set(reaction.products): # this is a potential reaction path
            if reaction.used: # we have used this reaction already and it was reachable
                return True
            # a potential path to synthesize this target, it is a working path if we can get all reagents:
            elif all(find_reaction(reagent, parsed_reactions) for reagent in reaction.reagents):
                reaction.used = True # tag this reactions as used in the synthesis
                return True

    return False
```

Reaction Pathway Analyzer: Two solutions

```
# Long Chain Synthesis
starting_materials = ["Fe", "O2", "C"]
reactions = [
    "4Fe + 3O2 → 2Fe2O3",
    "2C + O2 → 2CO",
    "Fe2O3 + 3CO → 2Fe + 3CO2",
    "CO2 + C → 2CO"
]
target = "CO2"

find_synthesis_path_broad(starting_materials, reactions, target)
find_synthesis_path_deep(starting_materials, reactions, target)
# Expected: Should find the multi-step path involving iron oxide formation and reduction
✓ 0.0s
```

Target can be synthesized.

$\text{Fe}_2\text{O}_3 + 3\text{CO} \rightarrow 2\text{Fe} + 3\text{CO}_2 \iff 2\text{C} + \text{O}_2 \rightarrow 2\text{CO} \iff 4\text{Fe} + 3\text{O}_2 \rightarrow 2\text{Fe}_2\text{O}_3 \iff \text{Stock}$

```
(True,
 ['→Fe+O2+C',
  '4Fe + 3O2 → 2Fe2O3',
  '2C + O2 → 2CO',
  'Fe2O3 + 3CO → 2Fe + 3CO2'])
```

Questions? Comments? What did you learn?



Let's try the next one!

Your assignment is in session04_assignment.

What you don't finish today in class will be **homework**. Ask questions and help each other.

Next session (Wednesday), please raise questions that came up while attempting the homework.

We will discuss solutions in class next Friday. The **homework must be submitted by 7:59 am next Friday**.

Submit homework to dimitrios.xynos@chem.uzh.ch, subject "DSA103_26_Ex1_LastName_FirstName".



<https://xkcd.com/1487/> 